



Ensemble Methods

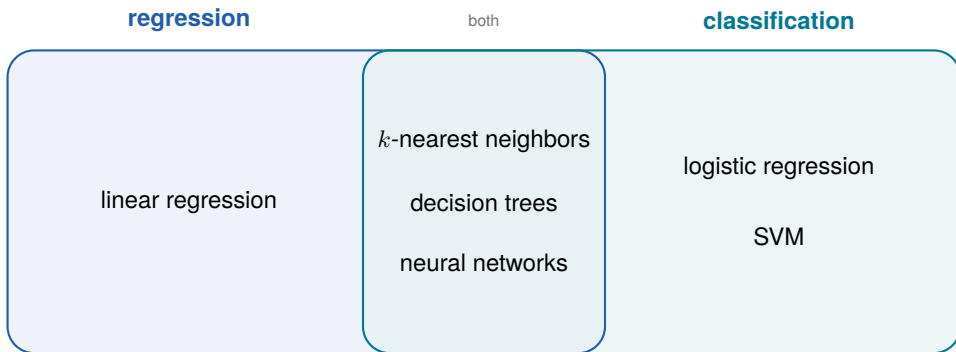
Why trust a single model?

Deli Lin

July 11, 2026

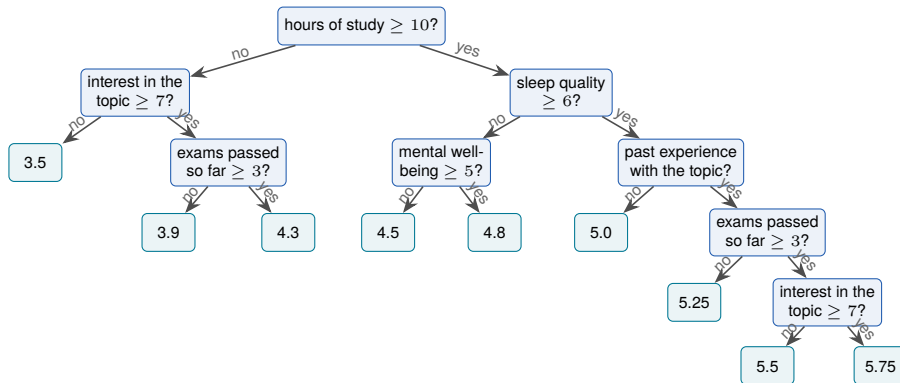


So far, we have learned many different models



What is a decision tree?

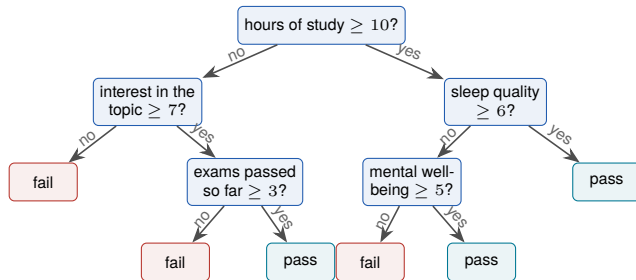
A sequence of yes/no questions about the features; each answer narrows down the predicted grade.



depth = 5: at most five questions from the top to a predicted grade

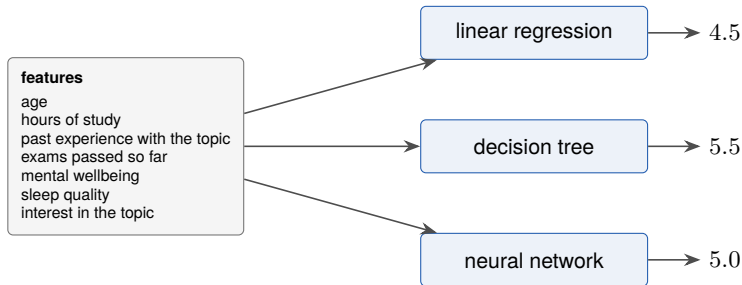
A tree can also classify

Same features, different target: will I pass the exam?



the leaves now hold a class (pass / fail) instead of a number

An example: predicting my grade in an exam



Three models, three different predictions.

The most intuitive combination

$$\frac{4.5 + 5.5 + 5.0}{3} = 5.0$$

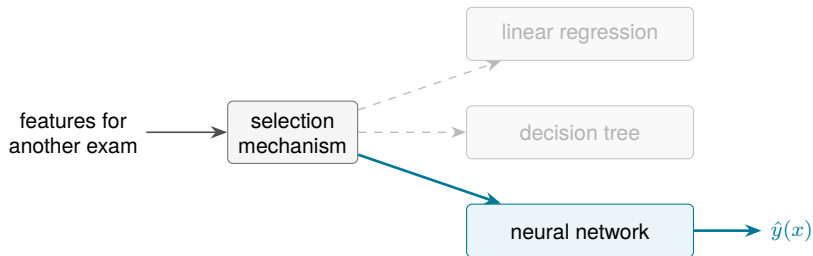
Ensemble Methods

Multiple models + a rule for combining their judgments.

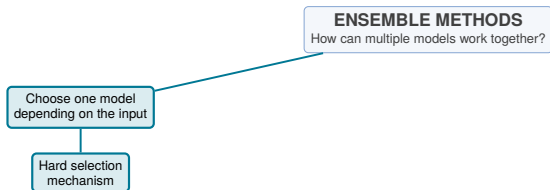
ENSEMBLE METHODS

How can multiple models work together?

Only one model answers

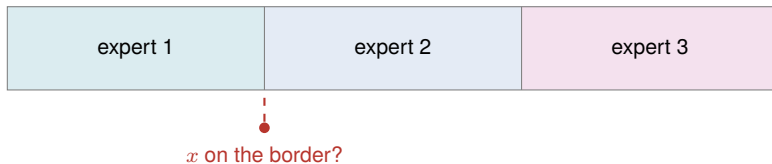


Only one model is active for a given input.



Why choose only one expert?

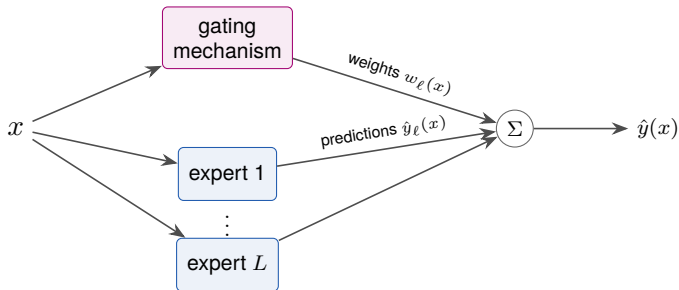
What if several models are useful for the same input?

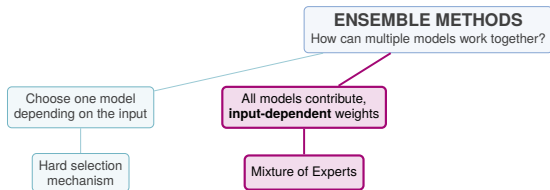


Mixture of Experts

All models contribute, weighted by a **linear combination** whose weights $w_\ell(x)$ depend on the input:

$$\hat{y}(x) = w_1(x) \hat{y}_1(x) + \cdots + w_L(x) \hat{y}_L(x)$$





Global combination

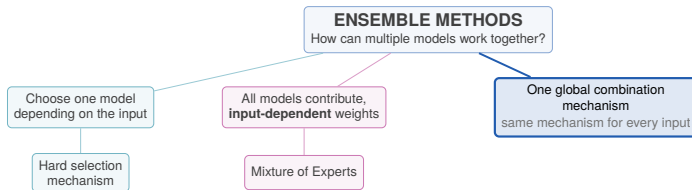
Use the same combination rule for every input.

before: the weights depend on the input

$$\hat{y}(x) = w_1(x) \hat{y}_1(x) + \cdots + w_L(x) \hat{y}_L(x)$$

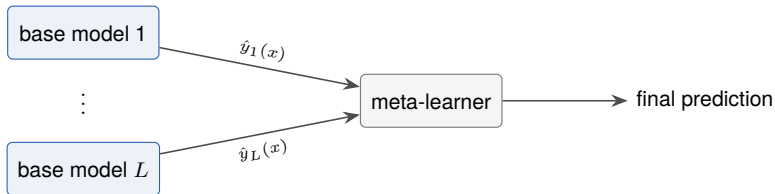
now: the same weights for every input

$$\hat{y}(x) = w_1 \hat{y}_1(x) + \cdots + w_L \hat{y}_L(x)$$



Stacking

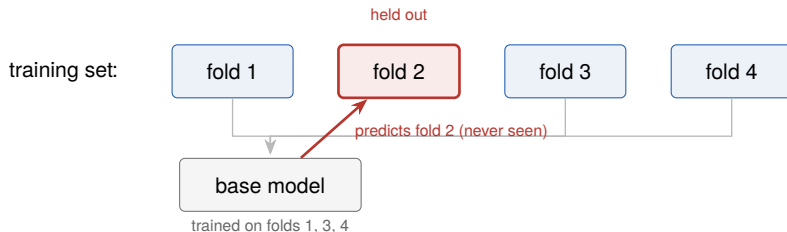
Train another model using the base-model predictions as inputs.



base models: the individual models whose judgments we combine

Out-of-fold predictions

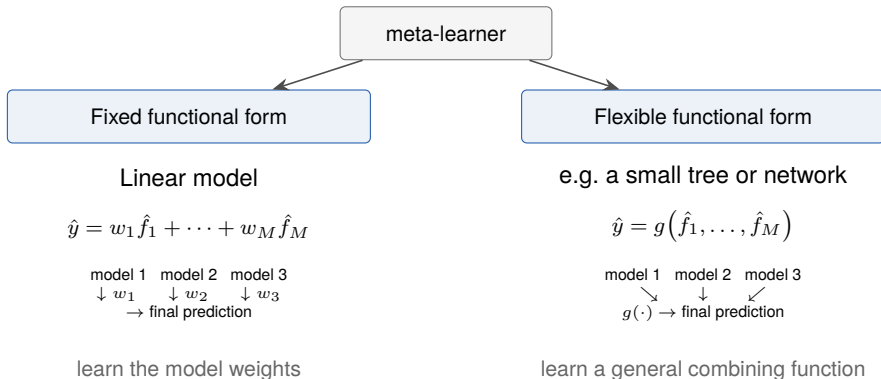
Train the meta-learner on predictions it could not have cheated on.



Risk: giving an unfair advantage to flexible models.

Solution: assign the weights by cross-validation, on out-of-fold predictions.

What can the meta-learner be?



Stacking in code

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor

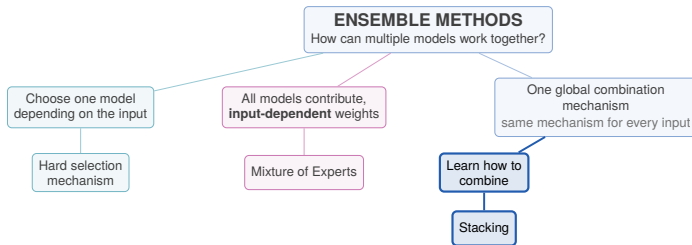
stack = StackingRegressor(
    estimators=[("lin", LinearRegression()),
                ("tree", DecisionTreeRegressor(max_depth=5)),
                ("net", MLPRegressor())],
    final_estimator=LinearRegression(positive=True), # linear meta-learner
    cv=5)                                           # out-of-fold predictions

stack.fit(X_train, y_train)

w = stack.final_estimator_.coef_
w = w / w.sum()                                # weights now sum to 1
```

cv=5: the meta-learner sees only out-of-fold predictions.

positive=True + rescaling: global model weights that sum to 1.



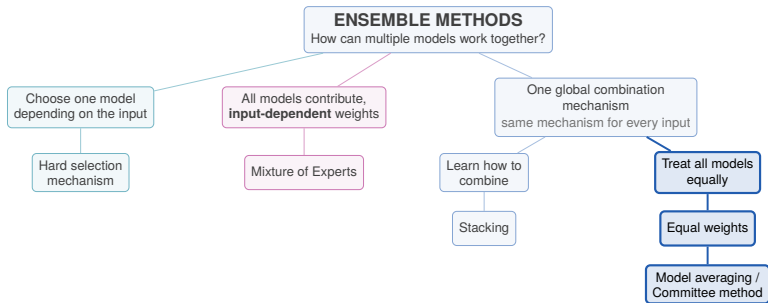
Treat all models equally

Give every model the same weight.

$$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^M \hat{f}_m(x)$$

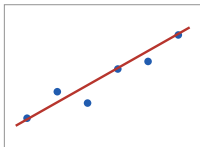
MODEL AVERAGING

also called the **committee method**

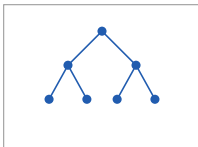


Where do the models come from?

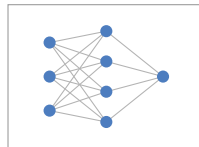
Different model families



linear model

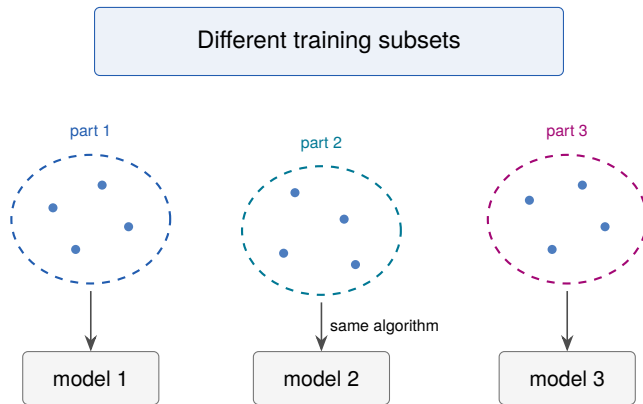


decision tree



neural network

Where do the models come from?



but: each model sees less data

Where do the models come from?

Different hyperparameters

```
from sklearn.tree import DecisionTreeRegressor

models = []
for depth in [2, 5, 10, 20]:
    tree = DecisionTreeRegressor(max_depth=depth)
    tree.fit(X_train, y_train)
    models.append(tree)
```

Same algorithm, same data: four different trees.

Where do the models come from?

Different model families

Different training subsets

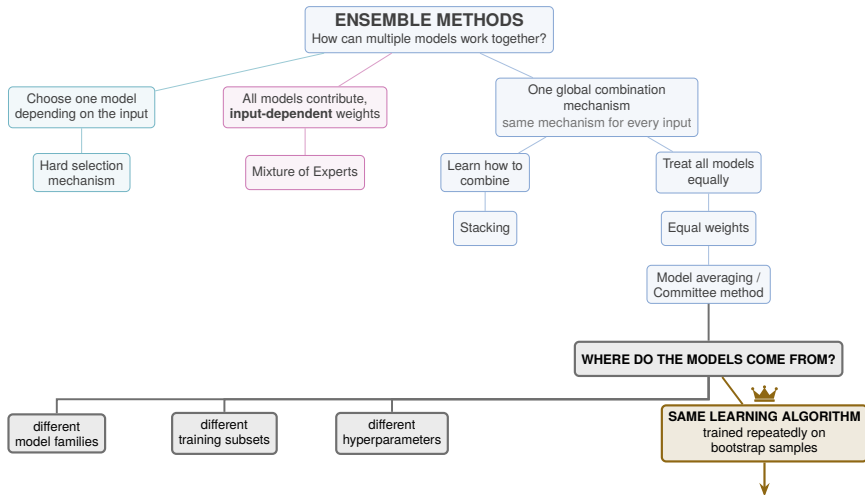
Different hyperparameters

Slightly different versions of the same dataset

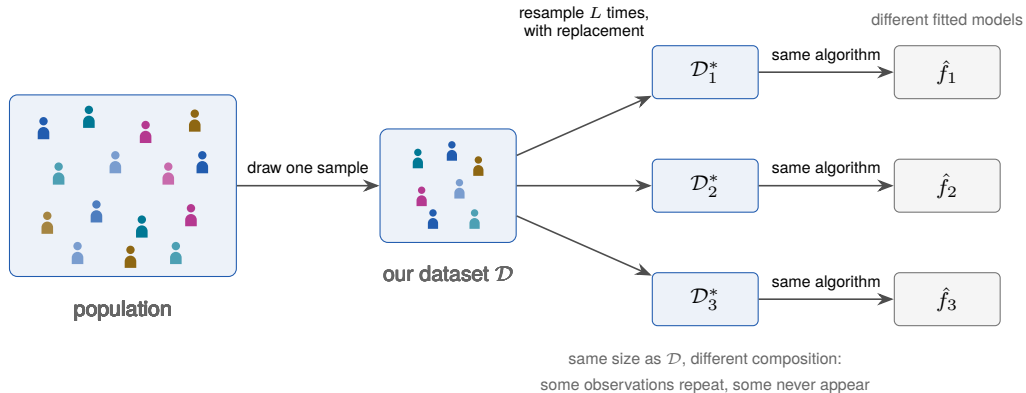




**TAKE THE SAME LEARNING ALGORITHM
AND TRAIN IT REPEATEDLY
ON SLIGHTLY DIFFERENT VERSIONS OF THE DATA**



Bootstrap



Bootstrapping creates diversity.

Bootstrap + Aggregating

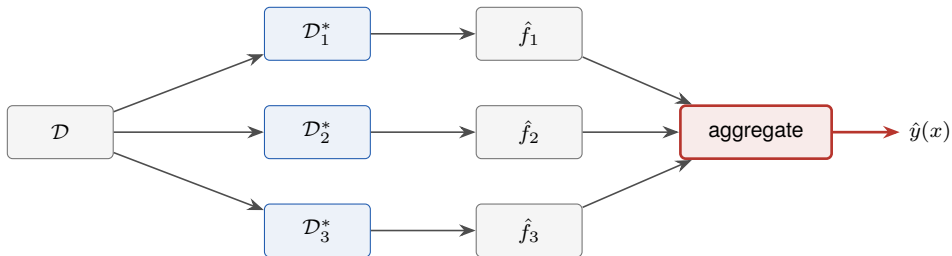


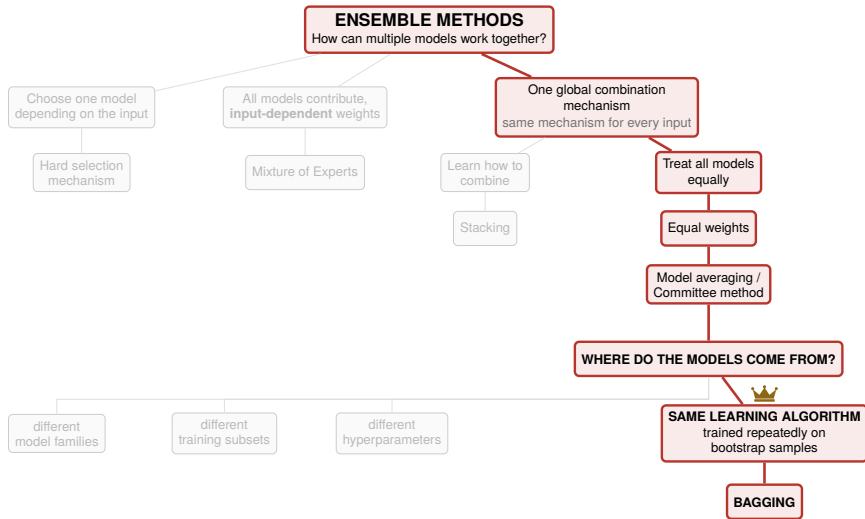
BAGGING

① bootstrap

② train

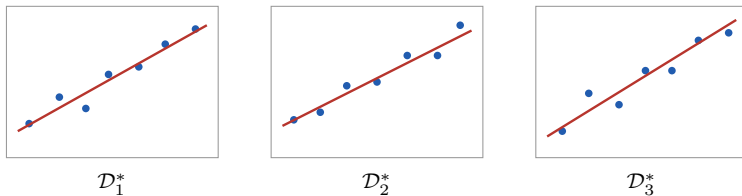
③ aggregate





Bagging linear regression

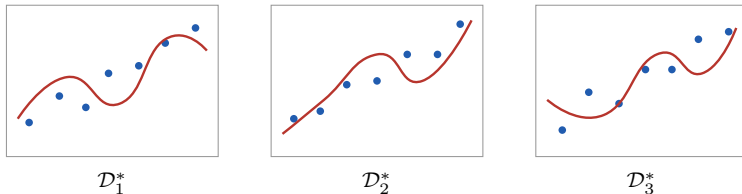
$\mathcal{D}_1^*, \mathcal{D}_2^*, \mathcal{D}_3^*$: bootstrap samples of $\mathcal{D} \rightarrow$ one fitted line each



three almost identical lines

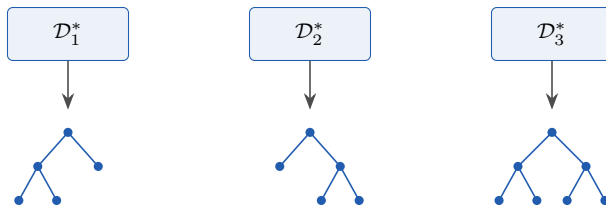
Bagging neural networks

same bootstrap samples $\rightarrow$ one fitted network each



three noticeably different curves

Bagging trees



slightly different bootstrap samples $\rightarrow$ different trees

Three candidates

bagged
linear regression

bagged
neural networks

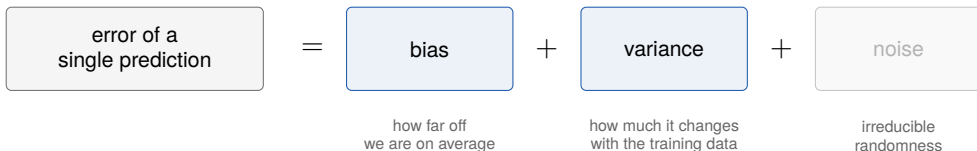
bagged
trees

Which one should we use? Keep them in mind.

Where does the error of a prediction come from?

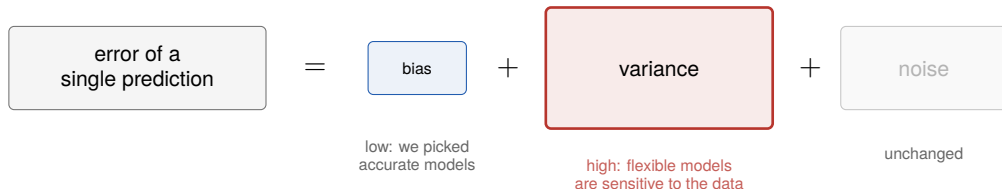
regression

The error of a single prediction has three parts.



We can do nothing about the noise: **we can control the bias and the variance.**

Pick very accurate models: the bias shrinks, the variance grows.

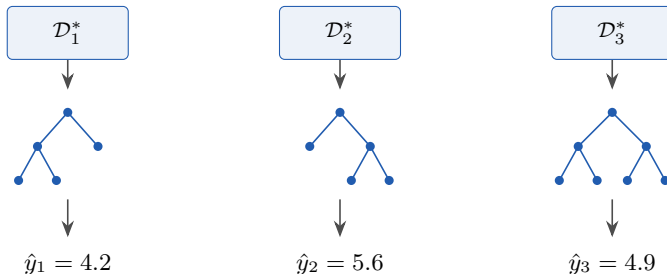


The total error can end up the same as before.
Keep the bias low, **and control the variance.**

Trees: low bias, high variance

regression

slightly different datasets $\rightarrow$ very different trees $\rightarrow$ very different predictions

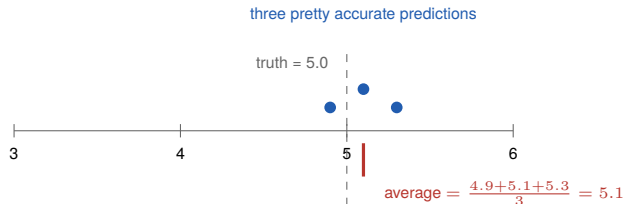


The variance of a model: how sensitive it is to the data it happens to train on.

Deep trees: **low bias**, but **high variance**.

How can I have something that keeps the bias low
and does not let the variance blow up?

Back to the grade example: my true grade is 5.0.



Accurate predictions average into an accurate prediction:
the bias is untouched.

VARIANCE REDUCTION

The answer: take the mean

regression

The mean keeps the bias untouched, **but lowers the variance**.

intuitively: relying on many judgments is less risky than relying on one



Bagging works best when the base learner is **powerful** but **unstable**.

Why is the average less variable?

regression

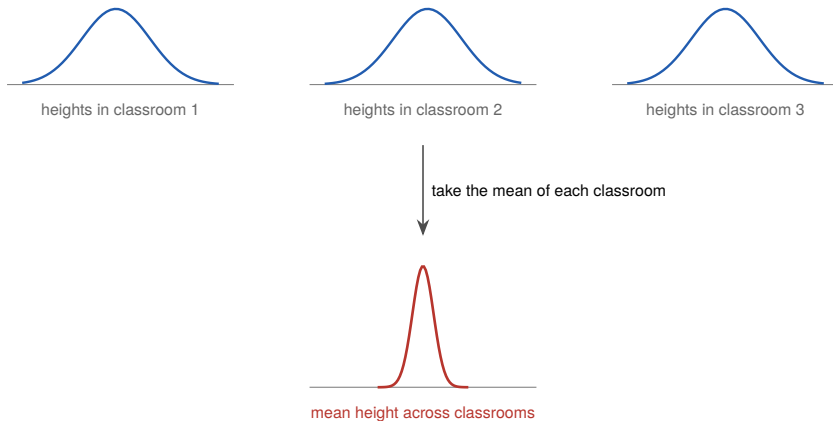
For i.i.d. predictions $\hat{Y}_1, \dots, \hat{Y}_L$ with variance σ^2 :

$$\text{Var}\left(\frac{1}{L} \sum_{\ell=1}^L \hat{Y}_{\ell}\right) = \frac{\sigma^2}{L} \leq \sigma^2$$

The mean is less variable than a single judgment.

The height intuition

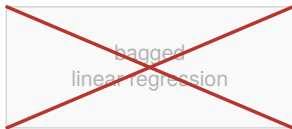
regression



Individual heights vary a lot. **Classroom means vary much less.**

Back to our three candidates

regression



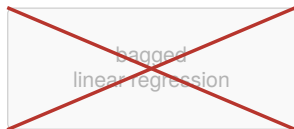
bagged
neural networks

bagged
trees

nearly the same line on every bootstrap sample:
shared systematic bias, almost no diversity to average away

Trees or neural networks?

regression



bagged
neural networks

unstable, but slow to train

bagged
trees

unstable, and fast to train

We usually prefer bagged trees.

We cannot simply average class labels.

Hard voting

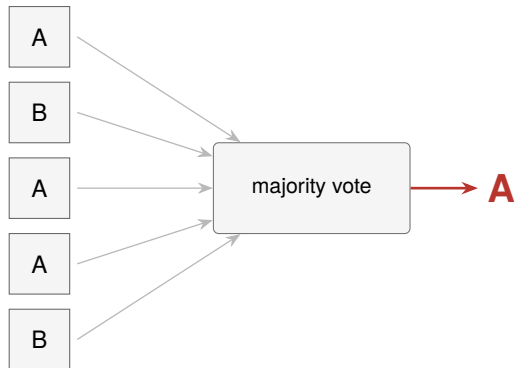
each classifier contributes
one class label

Soft voting

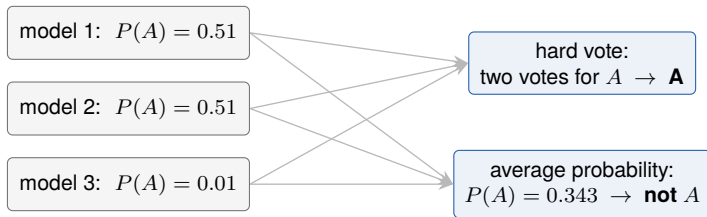
average the predicted
probabilities

Hard voting: majority vote

classification



They can disagree



Hard voting uses the final decisions.
Probability averaging also uses confidence.

The price we pay



single tree

interpretable



bagged ensemble

less interpretable

A bagged tree is no longer a tree.

The error of the mean prediction

before: a single prediction

$$\text{error of a single prediction} = \text{bias} + \text{variance} + \text{noise}$$

now: the mean prediction

$$\text{error of the mean prediction (bagging)} = \text{bias} + \text{variance} \downarrow + \text{noise}$$

low: averaging
keeps it low

bagging tries
to lower it

unchanged

The mean keeps the bias low and **tries to lower the variance.**

Are the trees independent?

The bias of deep trees is negligible: we focus on the variance.
earlier, for i.i.d. predictions:

$$\text{Var}\left(\frac{1}{L} \sum_{\ell=1}^L \hat{Y}_{\ell}\right) = \frac{\sigma^2}{L}$$

Trees grown on bootstrap samples of the *same* dataset
are **correlated**.

The variance of the average

identically distributed predictions, variance σ^2 , pairwise correlation ρ :

$$\text{Var}\left(\frac{1}{L} \sum_{\ell=1}^L \hat{Y}_{\ell}\right) = \underbrace{\rho \sigma^2}_{\text{does not vanish}} + \underbrace{\frac{1-\rho}{L} \sigma^2}_{\rightarrow 0 \text{ as } L \text{ grows}}$$

The lower the correlation between the models,

the more variance reduction we achieve.

Why are bagged trees correlated?

The same strong features dominate every tree.

similar first splits in every tree

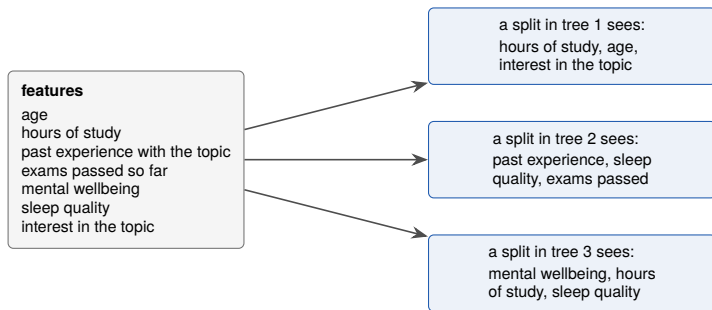


similar trees $\rightarrow$ correlated errors

Also sample the features

After bootstrapping the observations,
at every split the tree sees only a random subset of the features.

Back to the grade example



Every split sees only some of my features → the trees become less similar.

bootstrap the observations + sample the features

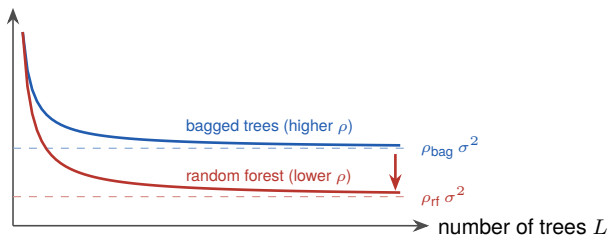


RANDOM FOREST

Random forest lowers the plateau

$$\text{Var}\left(\frac{1}{L} \sum_{\ell=1}^L \hat{Y}_{\ell}\right) = \underbrace{\rho \sigma^2}_{\text{we work on this term}} + \underbrace{\frac{1-\rho}{L} \sigma^2}_{\rightarrow 0 \text{ as the number of bagged models } L \text{ grows}}$$

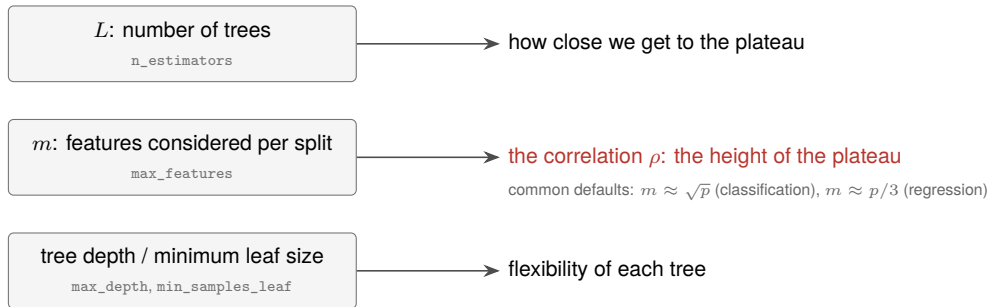
variance of the average



More trees only take you *to* the plateau.

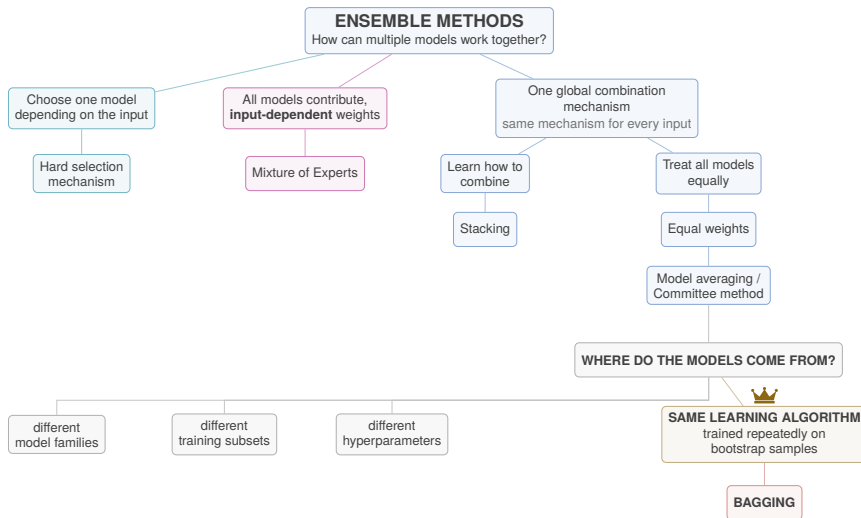
Lower correlation lowers the plateau itself.

Hyperparameters of a random forest



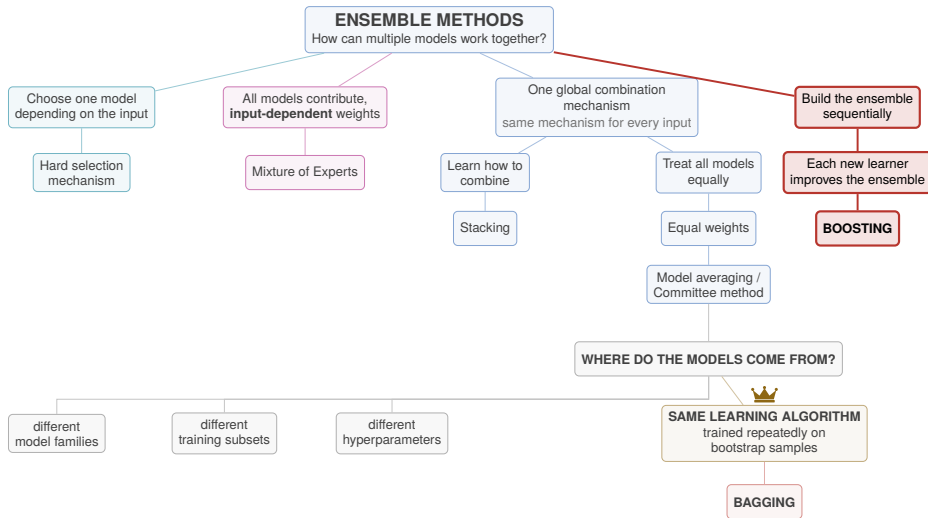
in scikit-learn: `sklearn.ensemble.RandomForestClassifier / RandomForestRegressor`

Recap



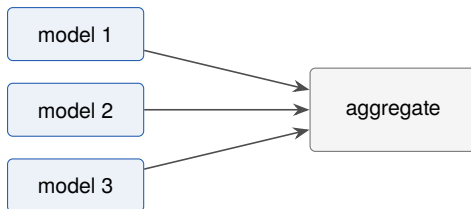
BOOSTING

From independent models to sequential correction



A different way to build an ensemble

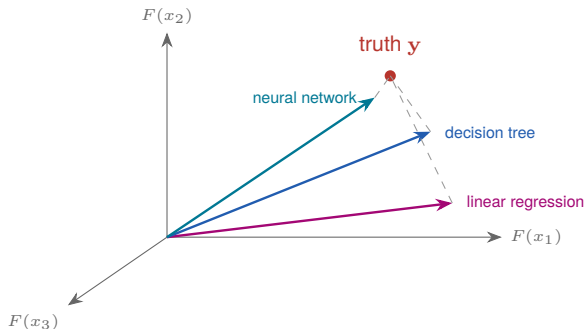
In bagging, the models can be trained independently.



Think only about its predictions

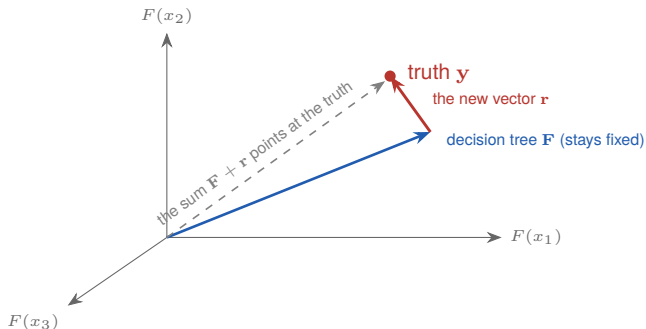
A model corresponds to a prediction vector on the training data.

$$\mathbf{F} = (F(x_1), F(x_2), \dots, F(x_n))^T$$



one axis per training observation: the space is n -dimensional; three axes are the best we can draw

What vector should we add?



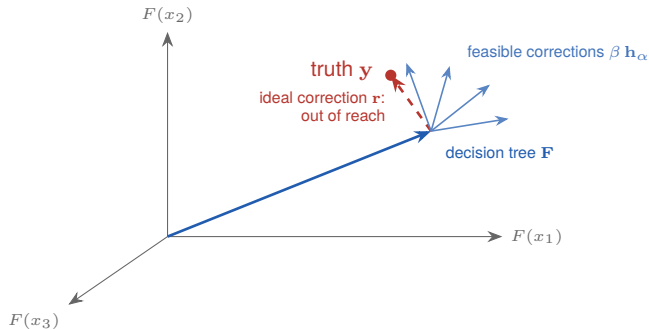
stagewise: the vector we already have stays fixed

additive: we add a new vector on top

the vector we want to add is **the residual vector r**

Why not simply add the residual vector?

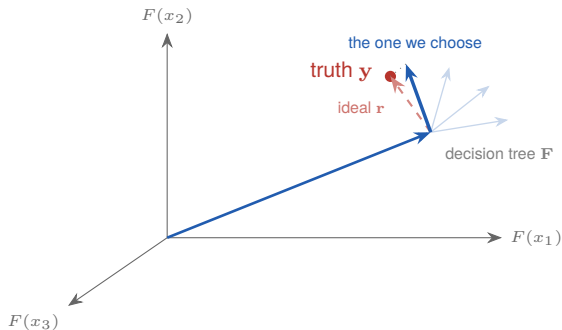
The next correction must be produced by our base learner: $\beta \mathbf{h}_\alpha$ α : internal parameters



No feasible vector is exactly $\mathbf{r}$: **we can only approximate it.**

pick the base learner whose prediction vector best approximates the correction we need

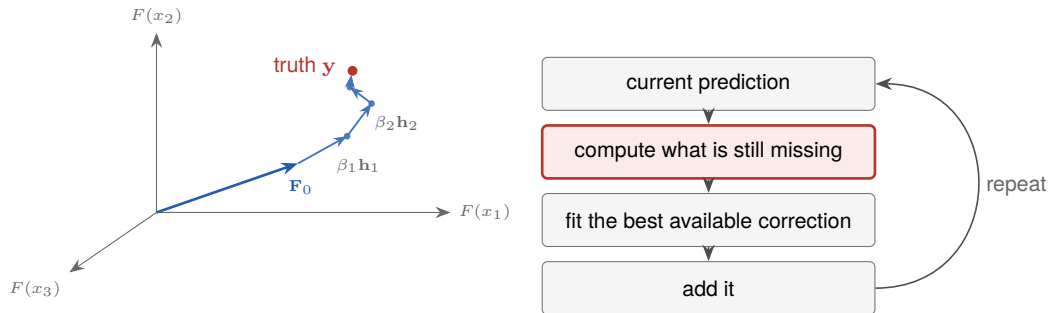
The correction we choose



Among the feasible vectors, we pick the one that ends **closest to y** .

Each step reacts to what is still wrong

$$F_m(x) = F_{m-1}(x) + \beta_m h_m(x)$$



Each new vector **reacts to what was still wrong** after the previous step.

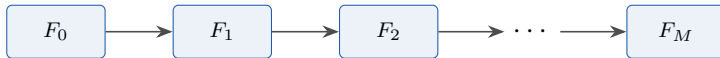
What have we just done?

What we have just done is called

Forward Stagewise Additive Modelling

many boosting algorithms fit this pattern

Forward



We move forward one stage at a time.

Stagewise

$$F_m(x) = F_{m-1}(x) + \text{new correction}$$

Previous components remain fixed.

Additive

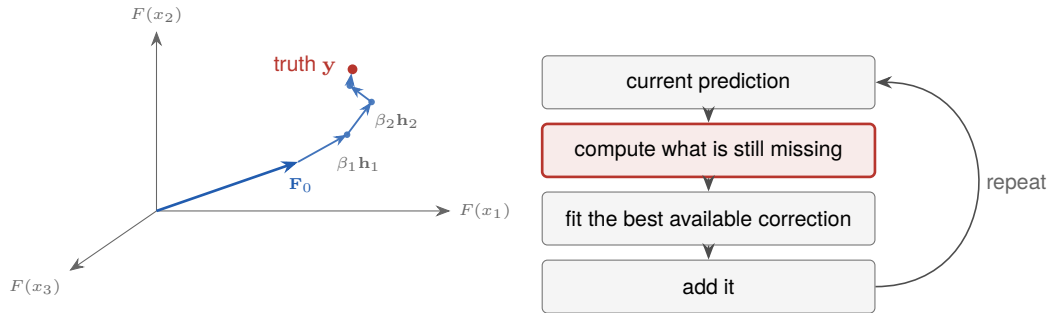
$$F_M(x) = F_0(x) + \sum_{m=1}^M \beta_m h_m(x)$$

F_0 : initial prediction h_m : new base learner β_m : contribution of that learner

This is still an **ensemble**: we combine the predictions,
just with a different approach.

Each step reacts to what is still wrong

$$F_m(x) = F_{m-1}(x) + \beta_m h_m(x)$$

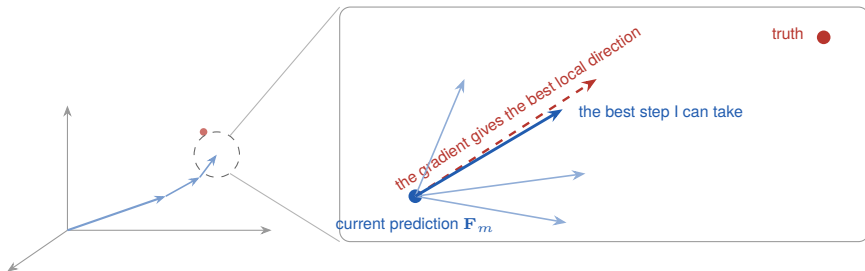


Each new vector **reacts to what was still wrong** after the previous step.

The loss that measures how far I am from the truth is, in this case, **the MSE**.

What if we change the loss?

At each step the question is local: in which direction should I step to reach the perfect prediction?



the best feasible vector: the closest to the direction suggested by the gradient found by minimizing the MSE between the gradient and the candidate step

Now that I am here, what is the best action to produce a new prediction that lowers the error?

The same question as gradient descent

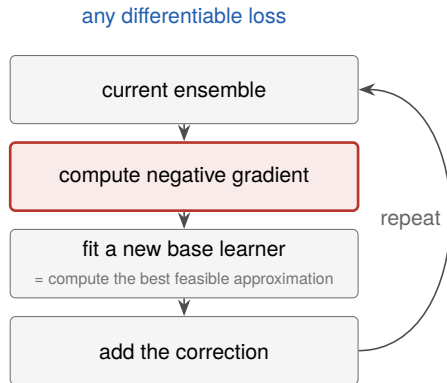
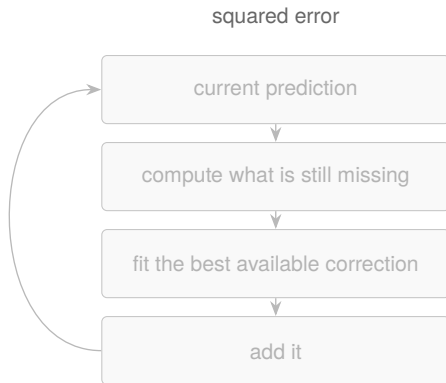
Now that I am here, what is the best **action** to produce a new prediction that **lowers the error**?

compare with

Now that I am here, what is the best **direction** to go to find the **minimum** of this landscape?

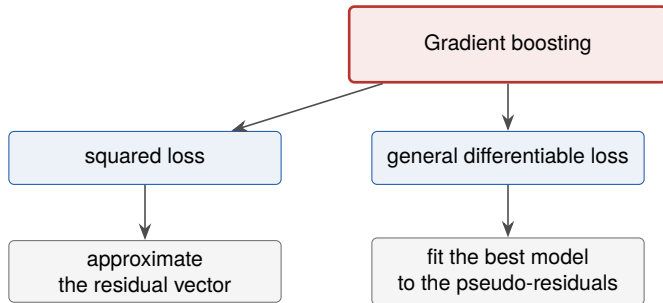
Gradient boosting is
gradient descent in function space.

Gradient boosting

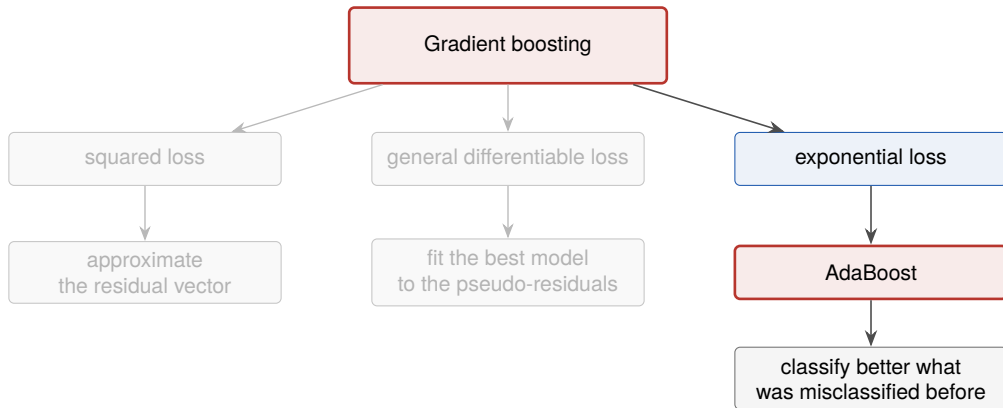


The same loop. We are simply approximating **the negative gradient**.

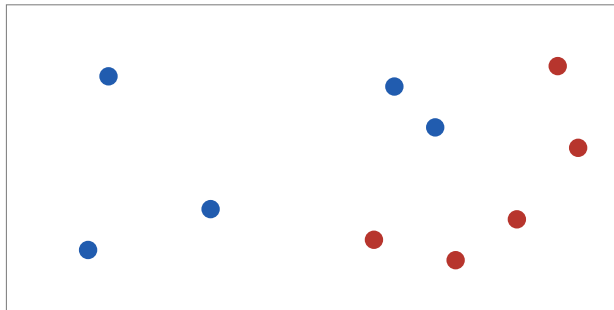
One framework



One framework

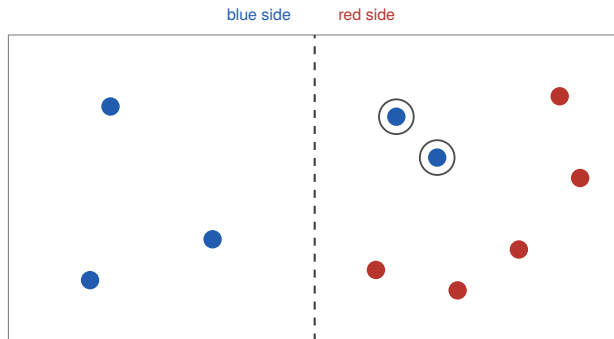


AdaBoost, step by step



Start: every observation has the same weight.

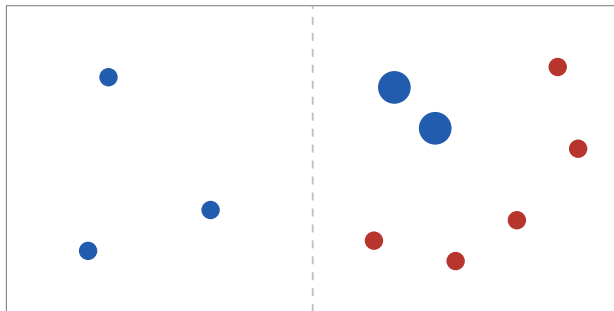
Weak classifier 1



A first simple split: two blue points end up on the wrong side.

ensemble so far: $F(x) = \alpha_1 h_1(x)$ α_1 : how much say this classifier gets in the final vote

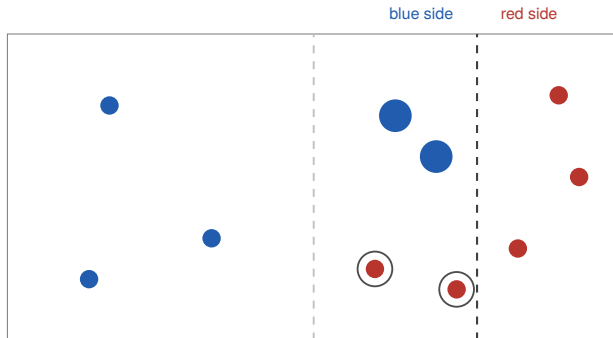
The mistakes gain weight



Misclassified observations are re-weighted more heavily.

ensemble so far: $F(x) = \alpha_1 h_1(x)$

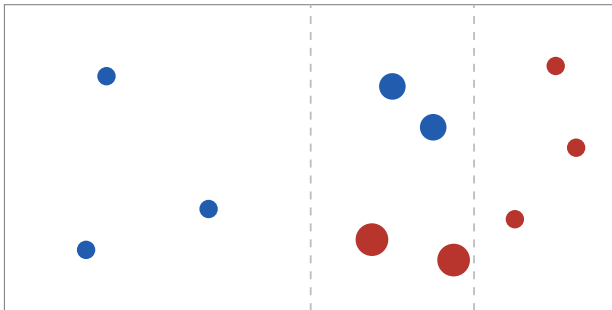
Weak classifier 2



Trained on the re-weighted data: the heavy blues are now correct, two reds are not.

ensemble so far: $F(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x)$

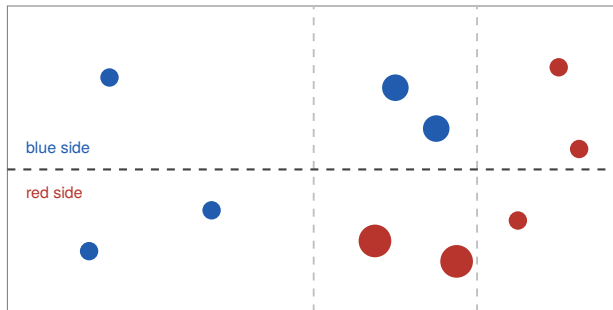
Re-weight again



The new mistakes grow; the corrected points shrink a little.

ensemble so far: $F(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x)$

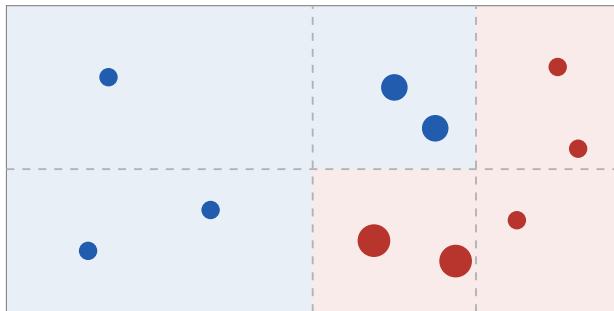
Weak classifier 3



A third split separates the heavy points; its own mistakes carry little weight.

ensemble so far: $F(x) = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x)$

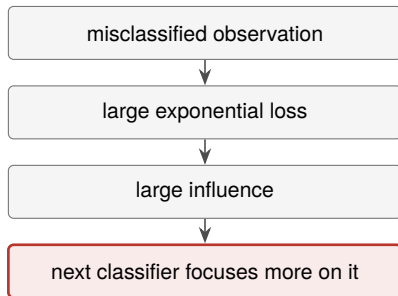
The strong classifier



The weighted vote of the three weak classifiers: every point is on the right side.

strong classifier: $\text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x))$

Why does AdaBoost focus on the mistakes?



Two interpretations

Algorithmic view
reweight difficult observations

Optimization view
follow the geometry of exponential loss

Why use restricted base learners?

A perfect learner could fit the whole residual in one step:

$$\mathbf{h}_m = \mathbf{r}_m \quad \implies \quad \text{training error} = 0$$

But the residual also contains **noise**,
and one big jump learns the noise too.

Make gradual changes at every step.

weak learner does not mean deliberately bad: it means controlled capacity

Regularizing boosting

1. Limit base-learner complexity

shallow trees, few leaves

`max_depth, max_leaf_nodes`

2. Shrink every correction

$$F_m = F_{m-1} + \nu \beta_m h_m, \quad 0 < \nu < 1$$

`shrinkage / learning_rate`



XGBoost, LightGBM, CatBoost

the same core idea, three modern implementations of gradient-boosted trees

XGBoost

first- and second-order
loss information
strong explicit regularization
highly optimized, parallel

`xgboost`

LightGBM

histogram-based splits
leaf-wise tree growth
very fast on large datasets

`lightgbm`

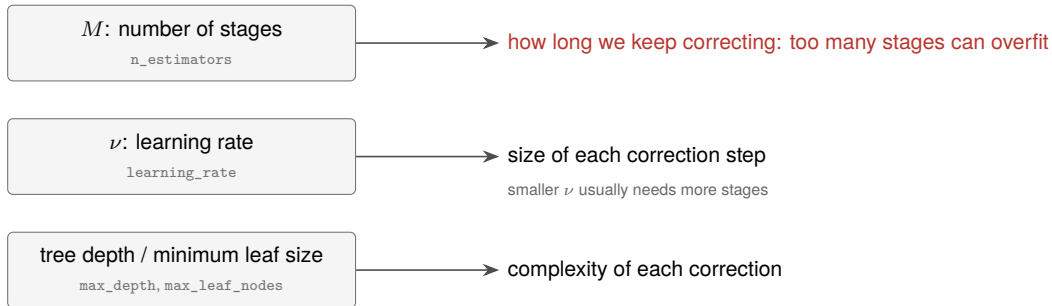
CatBoost

native categorical features
ordered boosting
against overfitting

`catboost`

I will leave the rest to your curiosity.

Hyperparameters of boosting



Unlike bagging: in boosting, adding more stages can overfit.